

SECURITY REVIEW

FOR GLUON



INSIDE THIS REPORT

CONTENTS

01	AUDIT SUMMARY	03
02	SCOPE	04
03	AUDITORS	05
04	METHODOLOGY	06
05	RISK CLASSIFICATION	07
06	FINDINGS SUMMARY	08
07	DETAILED FINDINGS	09
	M-01 Missing Validation of deposit to targetvault Return Value	09
	I-01 Redundant Balance Check and Math.min in deployFunds	10
08	DISCLAIMER	11
09	ABOUT KANN AUDITS	12

AUDIT SUMMARY

Gluon engaged Kann Audits to perform a security review of the Gluon smart contracts. A team of 3 security researchers performed a manual review of the in-scope contracts. All findings identified during the assessment are documented in this report.

TOTAL FINDINGS

2

ISSUES

CRITICAL	-
HIGH	-
MEDIUM	01
LOW	-
INFORMATIONAL	01

ENGAGEMENT DETAILS

PREPARED FOR	Gluon
PREPARED BY	Kann Audits
DATE REPORTED	19 September 2025
METHODS	Manual review

02 - WHAT WE REVIEWED

SCOPE

REPOSITORY	Private
AUDITED COMMIT	Private

#	FILE
01	/src/Forwarder.sol

03 - THE TEAM ON THIS ENGAGEMENT

AUDITORS

KA

Kann

LEAD SECURITY RESEARCHER

RA

Radev_eth

LEAD SECURITY RESEARCHER

FI

Fitro

LEAD SECURITY RESEARCHER

METHODOLOGY

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

STEP BY STEP

OUR SECURITY PROCESS

01

Kickoff & Context

A kickoff call to understand architecture, trust boundaries, threat assumptions and intended behaviour.

02

Scope & Preparation

Confirm the commit, files in scope, dependencies and the riskiest areas. We open a private channel between your team and our auditors.

03

AI First Scan

Kann AI, our proprietary model built to find security bugs, scans the codebase before the manual review starts.

04

Expert Security Review

Line-by-line analysis of exploit paths, invariant breaks, edge cases, accounting and integration risks.

05

Findings & Proofs

Each issue is confirmed, rated and backed by a proof of concept or a precise reproduction path.

06

Mitigation Review

We review every fix, confirm it resolves the root cause and flag partial or incorrect mitigations.

COVERAGE

THE REVIEW SPECIFICALLY CONSIDERED

- ✓ Testing against common and uncommon attack vectors
- ✓ Contract logic matching the client's intended behaviour
- ✓ Manual line-by-line review by experienced auditors
- ✓ Architecture and design analysis for secure component interactions and clear trust boundaries
- ✓ Compliance with industry best practices and known smart contract security standards
- ✓ Cross-referencing patterns with industry-standard implementations
- ✓ Access control verification
- ✓ State transitions and logic, to detect inconsistencies, unexpected behaviour or exploitation paths

RISK CLASSIFICATION

Severity combines the **impact** of an issue if exploited with the **likelihood** that the conditions for exploitation are met.

● CRITICAL	Directly exploitable with few or no preconditions, leading to theft or permanent loss of funds or loss of control of the system. Must be fixed before deployment.
● HIGH	Directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.
● MEDIUM	Moderate loss of funds or impact on multiple users, but only under specific conditions. Exploitable without privileged access, but typically requires certain assumptions or edge cases.
● LOW	Issues with limited impact that do not put funds directly at risk, such as edge cases or deviations from documented behaviour.
● INFORMATIONAL	Non-exploitable findings such as best-practice deviations, cosmetic issues or documentation and compliance notes. They do not need urgent remediation but improve code quality and maintainability.

06 - ALL ISSUES

FINDINGS SUMMARY

● CRITICAL	● HIGH	● MEDIUM	● LOW	● INFO
-	-	01	-	01
TOTAL 02 FIXED 02 ACKNOWLEDGED 00				

ID	TITLE	SEVERITY	STATUS
M-01	Missing Validation of deposit to targetvault Return Value	● MEDIUM	● FIXED
I-01	Redundant Balance Check and Math.min in deployFunds	● INFORMATIONAL	● FIXED

DETAILED FINDINGS

M-01

Missing Validation of deposit to targetvault Return Value

SEVERITY

● MEDIUM

STATUS

● FIXED

■ DESCRIPTION

The `_deployFunds` function forwards funds to `targetVault` by calling:

```
targetVault.deposit(deployAmount, address(this));
```

The `_deployFunds` function assumes `targetVault.deposit(...)` will always give the strategy a non-zero amount of vault shares. In normal, well-behaved vaults that's true, but there are cases where this assumption can fail. For example, if a freshly launched/integrated vault has been pre-funded (someone transfers assets directly to the vault contract without calling `transfer` the vault's share-calculation can be skewed and a subsequent deposit call can result in 0 shares minted for the depositor. Other edge cases could also lead to the same outcome, leaving the strategy's assets effectively locked in the vault with no credited shares.

Recommendation Always validate that shares were actually received:

```
uint256 sharesReceived = targetVault.deposit(deployAmount, address(this));
require(sharesReceived > 0, "Deposit did not mint shares");
```

■ REMEDIATION

● FIXED

I-01

Redundant Balance Check and Math.min in deployFunds

SEVERITY

● INFORMATIONAL

STATUS

● FIXED

■ DESCRIPTION

A user calls deposit() This function is defined in TokenizedStrategy and is executed via delegatecall from the Forwarder (which inherits BaseStrategy). As a result, storage updates happen in the Forwarder, msg.sender remains the user, and address(this) refers to the Forwarder.

Inside TokenizedStrategy.deposit: Assets are transferred from the user to the Forwarder (`_asset.safeTransferFrom(msg.sender, address(this), assets)`). then `IBaseStrategy(address(this)).deployFunds(_asset.balanceOf(address(this)))`; We call `deployFunds` with the balance of the forwarder = `BaseStrategy` where then inside Forwarder it gets `uint256 deployAmount = Math.min(_amount, balance)`; Which both values are same since `_amount` is address this balance of asset and `balance` is `address(this)` balance of the asset in the contract.

`Math.min` and getting the balance of the contract in `_deployFunds` is redundant.

■ REMEDIATION

● FIXED

DISCLAIMER

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource and expertise-bound effort in which we aim to identify as many issues as possible.

This review covers only the files listed in the Scope section at the audited commit. Code changed after that commit, external dependencies, deployment configuration and operational practices may introduce additional risk.

ABOUT KANN AUDITS

Kann Audits is a Web3 security audit company helping DeFi protocols, blockchain networks, and infrastructure teams build securely at every stage, from development to launch and beyond.



EXPERT SECURITY AUDITS

Human-led smart contract security audits built around the architecture, business logic, integrations and risk profile of complex Web3 protocols.



AI SECURITY SCAN

Kann AI extracts Solidity call graphs and returns function-level security signals for expert validation.



FORMAL VERIFICATION

Define security properties for critical protocol behaviour and evaluate whether the implementation satisfies those specifications.



INCIDENT RESPONSE

A direct triage path for suspected exploits, critical vulnerabilities, suspicious transactions and urgent remediation review.



KANN AUDITS
WEB3 SECURITY AUDITS

GET IN TOUCH

WEBSITE	kannaudits.com
PORTFOLIO	github.com/KannAudits
EMAIL	contact@kannaudits.com
X	@KannAudits
TELEGRAM	@KannAudits